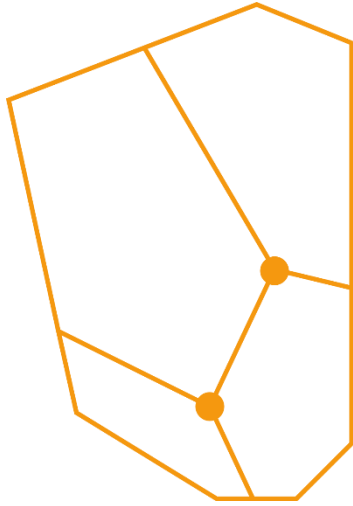




CAMPUS DIGITAL FP

CFGS Desarrollo de Aplicaciones Multiplataforma

Programación multimedia y dispositivos móviles

Practica RA3 PMDM03: Android avanzado. Mi rutina
fitness.

02 / 05 /2026

Raúl Gómez Sanz

Índice

Contenido

0.	Url repositorio. Mi rutina fitness y creación proyecto	2
1.	Configuración de rutas y pestañas	2
2.	Diseño de interfaz	3
3.	Implementación de lógica en TypeScript	5
4.	Persistencia de datos	6
5.	Estilo y diseño.....	7

Solución

0. Url repositorio. Mi rutina fitness y creación proyecto

HTTPS: https://github.com/The-Rul/android-learning_mi-rutina-fitness.git

CLI: `gh repo clone The-Rul/android-learning_mi-rutina-fitness`

```
import type { CapacitorConfig } from '@capacitor/cli';

const config: CapacitorConfig = {
  appId: 'com.campusdigitalfp.mirutinafitness',
  appName: 'mi-rutina-fitness',
  webDir: 'www'
};

export default config;
```

Figure 1. Configuración capacitor.config.ts

1. Configuración de rutas y pestañas

Configuración de rutas en el archivo `tabs-routing.module.ts`. e iconos en `tabs.page.html`.

```
const routes: Routes = [
  {
    path: 'tabs',
    component: TabsPage,
    children: [
      {
        path: 'rutinas',
        loadChildren: () => import(
      },
      {
        path: 'add-ejercicio',
        loadChildren: () => import(
      },
      {
        path: 'configuracion',
        loadChildren: () => import(
      },
      {
        path: '',
        redirectTo: '/tabs/rutinas',
        pathMatch: 'full'
      }
    ]
  },
  {
    path: '',
    redirectTo: '/tabs/rutinas',
    pathMatch: 'full'
  }
];
```

```
<ion-tabs>

  <ion-tab-bar slot="bottom">
    <ion-tab-button tab="rutinas" href="/tabs/rutinas">
      <ion-icon aria-hidden="true" name="barbell-outline"></ion-icon>
      <ion-label>Rutinas</ion-label>
    </ion-tab-button>

    <ion-tab-button tab="add-ejercicio" href="/tabs/add-ejercicio">
      <ion-icon aria-hidden="true" name="add-circle-outline"></ion-icon>
      <ion-label>Añadir Ejercicio</ion-label>
    </ion-tab-button>

    <ion-tab-button tab="configuracion" href="/tabs/configuracion">
      <ion-icon aria-hidden="true" name="settings-outline"></ion-icon>
      <ion-label>Configuración</ion-label>
    </ion-tab-button>
  </ion-tab-bar>
</ion-tabs>
```

Figure 2. Adaptación tabs.page.html

Figure 3. Archivo tabs-routing.module.ts

Se cambian los títulos de cada pestaña en `tabs1.page`, `tabs2.page` y `tabs3.page`

```
Go to component
<ion-header [translucent]="true">
  <ion-toolbar>
    <ion-title>
      Rutinas
    </ion-title>
  </ion-toolbar>
</ion-header>
```

Figure 4. Cambio titulo en tabs1.page.html

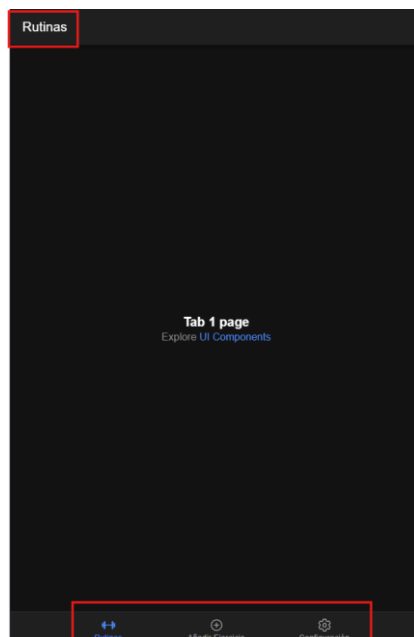


Figure 5. Funcionamiento correcto de rutas y nombres.

2. Diseño de interfaz

Se diseñan las 3 interfaces con elementos html ion: ion-list, ion-item...

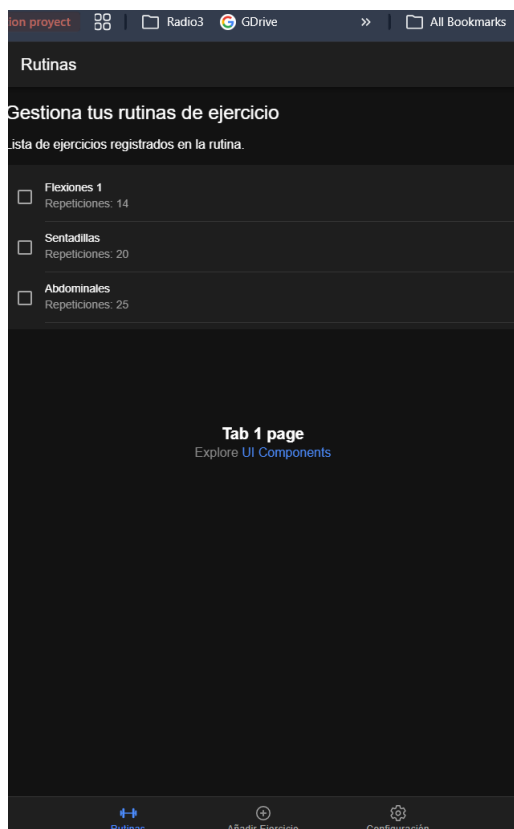


Figure 7. Interfaz tab1: rutina ejercicios

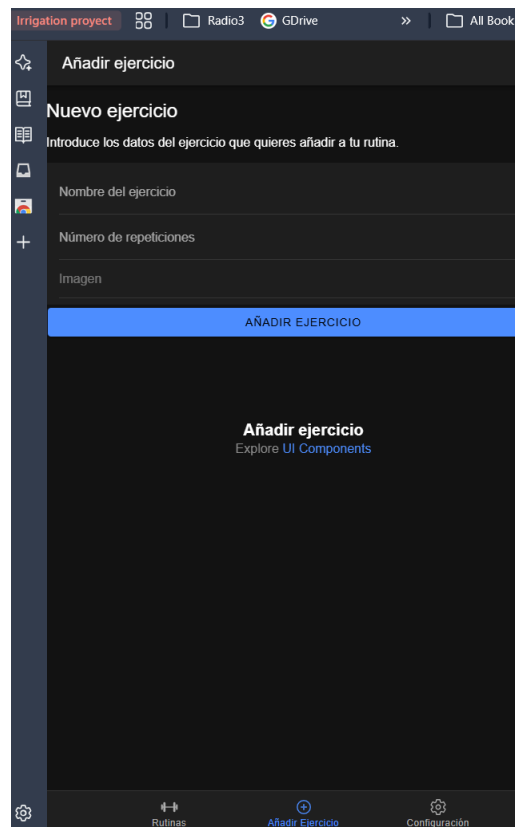


Figure 6. Interfaz tab2: añadir ejercicios

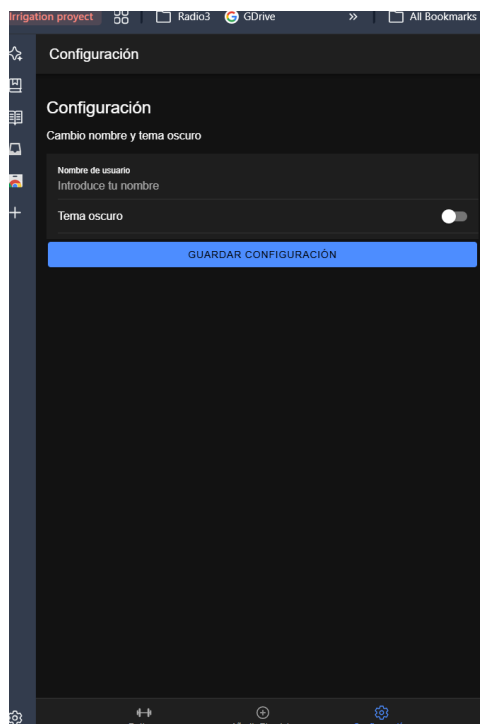


Figure 8. Interfaz tab3: configuración

3. Implementación de lógica en TypeScript

Se crea una interfaz *Ejercicio.model.ts* en la carpeta *src/app/models* para crear los ejercicios, identificarlos con un id y poder trabajar con ellos a lo largo de la aplicación. Y además una interfaz para guardar la configuración de usuario (nombre y tema).

```
src > app > models > TS ejercicio.model.ts > Ejercicio
1 export interface Ejercicio {
2   id: string;
3   nombre: string;
4   repeticiones: number;
5   completado: boolean;
6   imagen?: string;
7 }
```

Ilustración 9. Interfaz *ejercicio.model.ts*

```
export interface ConfiguracionUsuario {
  nombreUsuario: string;
  temaOscuro: boolean;
}
```

Ilustración 10. Interfaz *ConfiguracionUsuario*

Se centralizan los métodos CRUD de la lógica en el documento *rutina.service.ts*. Se añaden los principales métodos para el funcionamiento:

- *cargarEjercicios()*: carga la lista de ejercicios almacenados
- *guardarEjercicios()*: guarda una nueva lista con algún ejercicio modificado.
- *addEjercicios()*: pasando un nombre y el número de repeticiones llama crea un nuevo ejercicios y lo guarda.
- *cambiarEstadoEjercicio()*: pasando id y boolean cambia el estado del ejercicio a completado
- *eliminarEjercicios()*: pasando id de ejercicio lo borra de la lista persistente.,

```
async cargarEjercicios(): Promise<void> {
  try {
    const { value } = await Preferences.get({ key: this.EJERCICIOS_KEY });
    console.log('Datos crudos de preferencias: ', value);

    const ejercicios: Ejercicio[] = value ? JSON.parse(value) : []; // Parse
    this.ejerciciosSubject.next(ejercicios);

    console.log('Ejercicios cargados: ', ejercicios);
  } catch (error) {
    console.error('Error al cargar ejercicios:', error);
    this.ejerciciosSubject.next([]); // Si hay un error, se establece lista
  }
}
```

Ilustración 11. función *cargarEjercicios()*

```
* Guarda la lista actual de ejercicios en el almacenamiento
//
async guardarEjercicios(ejercicios: Ejercicio[]): Promise<void> {
  try {
    await Preferences.set({
      key: this.EJERCICIOS_KEY,
      value: JSON.stringify(ejercicios)
    });
    this.ejerciciosSubject.next(ejercicios); // Actualiza la lista
    console.log('Ejercicios guardados: ', ejercicios);
  } catch (error) {
    console.error('Error al guardar ejercicios:', error);
  }
}
```

Ilustración 12 función *guardarEjercicios()*

```
async addEjercicio(nombre: string, repeticiones: number): Promise<void> {
  const ejerciciosActuales = this.ejerciciosSubject.value;

  const nuevoEjercicio: Ejercicio = { // Se crea un ejercicio nuevo con los
    id: crypto.randomUUID(),
    nombre,
    repeticiones,
    completado: false,
  };

  const ejerciciosActualizados = [...ejerciciosActuales, nuevoEjercicio]; //
  await this.guardarEjercicios(ejerciciosActualizados); // Se guarda la nue
  console.log('Nuevo ejercicio añadido: ', nuevoEjercicio);
}
```

Ilustración 13 función *addEjercicio()*

```
async cambiarEstadoEjercicio(id: string, completado: boolean): Promise<void> {
  const ejerciciosActualizados = this.ejerciciosSubject.value.map((ejercicio) =>
    ejercicio.id === id // Si el id coincide, se crea nuevo objeto con
    ? { ...ejercicio, completado }
    : ejercicio
  );

  await this.guardarEjercicios(ejerciciosActualizados);

  console.log('Ejercicio ${id} actualizado. Completado: ', completado);
}
```

Ilustración 14. *cambiarEstadoEjercicio()*

Para aplicar estas funciones, se modifican los elementos pertinentes en el archivo *TypeScript* dedicado a la lógica de cada pestaña: *tab1.page.ts*.

En la pestaña principal, *tab1*, se carga la lista de ejercicios persistida al iniciar la aplicación y al volver a entrar en la pestaña. Además, mediante eventos asociados a botones, se llaman los métodos para cambiar el estado de los ejercicios y eliminarlos.

Desde los botones eliminar y checkbox del html se llama a los correspondientes métodos.

```

/**
 * Al iniciar sesión se carga la lista ejercicios en dispositivo.
 */
async ngOnInit(): Promise<void> {
  this.rutinaService.ejercicios$.subscribe((ejercicios) => {
    this.ejercicios = ejercicios;
  });

  await this.rutinaService.cargarEjercicios();
}

/**
 * Al entrar a la vista se recarga la lista actualizada
 */
async ionViewWillEnter(): Promise<void> {
  await this.rutinaService.cargarEjercicios();
}

/**
 * Recoge el evento y lo cambia en el ejercicio.
 */
async cambiarEstado(event: CustomEvent, id: string): Promise<void> {
  const completado = event.detail.checked;
  await this.rutinaService.cambiarEstadoEjercicio(id, completado);
}

```

Ilustración 15 llamada a lógica para *tab1.page.ts*

```

<ion-list *ngIf="ejercicios.length > 0; else sinEjercicios">
  <ion-item *ngFor="let ejercicio of ejercicios">
    <ion-checkbox
      slot="start"
      [checked]="ejercicio.completado"
      (ionChange)="cambiarEstado($event, ejercicio.id)"
    >
  </ion-checkbox>

  <ion-label>
    <h3>{{ ejercicio.nombre }}</h3>
    <p>Repeticiones: {{ ejercicio.repeticiones }}</p>
    <p>Estado: {{ ejercicio.completado ? 'Realizado' : 'No realizado' }}</p>
  </ion-label>

  <ion-button
    slot="end"
    fill="clear"
    color="primary"
    (click)="eliminarEjercicio(ejercicio.id)"
  >

```

Ilustración 16 acciones en botones *tab1.html*

Para el resto de pestañas se realiza de manera similar para los correspondientes métodos y en sus correspondientes archivos.

4. Persistencia de datos

La persistencia se ha implementado mediante el plugin *@capacitor/preferences*. Para ello, los *ejercicios* se convierten a formato JSON con *JSON.stringify()* antes de guardarse, y se recuperan con *JSON.parse()* al iniciar o volver a entrar en la pestaña de rutinas.

```

/**
 * Carga los ejercicios desde las preferencias.
 */
async cargarEjercicios(): Promise<void> {
  try {
    const { value } = await Preferences.get({ key: this.EJERCICIOS_KEY });
    console.log('Datos crudos de preferencias: ', value);

    const ejercicios: Ejercicio[] = value ? JSON.parse(value) : []; // Parseo a JSON
    this.ejerciciosSubject.next(ejercicios);

    console.log('Ejercicios cargados: ', ejercicios);
  } catch (error) {
    console.error('Error al cargar ejercicios: ', error);
  }
}

```

Ilustración 17 Cargar los ejercicios desde *preference*.

```

*/
async guardarEjercicios(ejercicios: Ejercicio[]): Promise<void> {
  try {
    await Preferences.set({
      key: this.EJERCICIOS_KEY,
      value: JSON.stringify(ejercicios)
    });
  }
}

```

Ilustración 18. Guardar los ejercicios con preference

Se hace de similar manera para los métodos de carga y guardado de la configuración.

5. Estilo y diseño.

Se realiza un diseño final con CSS con colores primario #1b5c70 y secundario específicos. #ffb703.

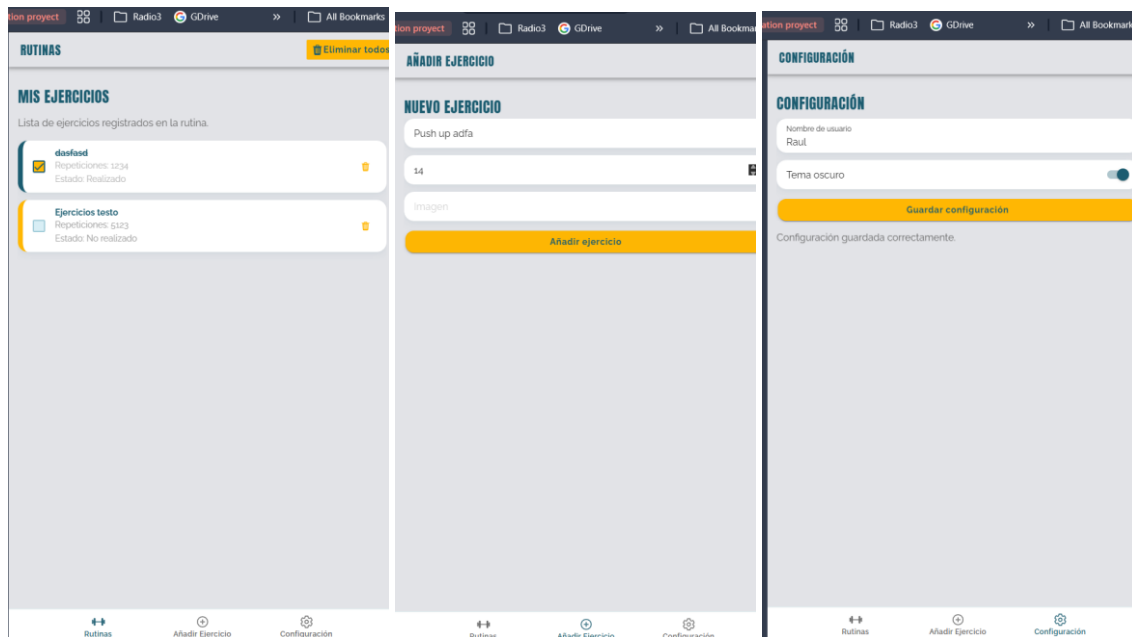


Ilustración 19. Diseño final aplicación ejercicios.